UiA University of Agder

Master

DEEP RESEARCH AGENT

Vishnu Vardhan Addanki Tirumala, Rémi Wysocka, Mary Asha Reddy Boreddy

SUPERVISOR
Prof. Morten Goodwin

University of Agder, 2026
Faculty of Engineering and Science
Department of Engineering and Sciences

Obligatorisk gruppeerklæring

Den enkelte student er selv ansvarlig for å sette seg inn i hva som er lovlige hjelpemidler, retningslinjer for bruk av disse og regler om kildebruk. Erklæringen skal bevisstgjøre studentene på deres ansvar og hvilke konsekvenser fusk kan medføre. Manglende erklæring fritar ikke studentene fra sitt ansvar.

1.	Vi erklærer herved at vår besvarelse er vårt eget arbeid, og at vi ikke har brukt andre kilder eller har mottatt annen hjelp enn det som er nevnt i besvarelsen.	Ja / Nei
2.	**Vi erklærer videre at denne besvarelsen:** • Ikke har vært brukt til annen eksamen ved annen avdeling/universitet/høgskole innenlands eller utenlands. • Ikke refererer til andres arbeid uten at det er oppgitt. • Ikke refererer til eget tidligere arbeid uten at det er oppgitt. • Har alle referansene oppgitt i litteraturlisten. • Ikke er en kopi, duplikat eller avskrift av andres arbeid eller besvarelse.	Ja / Nei
3.	Vi er kjent med at brudd på ovennevnte er å betrakte som fusk og kan medføre annullering av eksamen og utestengelse fra universiteter og høgskoler i Norge, jf. Universitets- og høgskoleloven §§4-7 og 4-8 og Forskrift om eksamen §§ 31.	Ja / Nei
4.	Vi er kjent med at alle innleverte oppgaver kan bli plagiatkontrollert.	Ja / Nei
5.	Vi er kjent med at Universitetet i Agder vil behandle alle saker hvor det forligger mistanke om fusk etter høgskolens retningslinjer for behandling av saker om fusk.	Ja / Nei
6.	Vi har satt oss inn i regler og retningslinjer i bruk av kilder og referanser på biblioteket sine nettsider.	Ja / Nei
7.	Vi har i flertall blitt enige om at innsatsen innad i gruppen er merkbart forskjellig og ønsker dermed å vurderes individuelt. Ordinært vurderes alle deltakere i prosjektet samlet.	Ja / Nei

Publiseringsavtale

Fullmakt til elektronisk publisering av oppgaven Forfatter(ne) har opphavsrett til oppgaven. Det betyr blant annet enerett til å gjøre verket tilgjengelig for allmennheten (Åndsverkloven. §2).
Oppgaver som er unntatt offentlighet eller taushetsbelagt/konfidensiell vil ikke bli publisert.

Vi gir herved Universitetet i Agder en vederlagsfri rett til å gjøre oppgaven tilgjengelig for elektronisk publisering:	Ja / Nei
Er oppgaven båndlagt (konfidensiell)?	Ja / Nei
Er oppgaven unntatt offentlighet?	Ja / Nei

Acknowledgements

We would like to express our deepest gratitude to our supervisors, Professor Morten Goodwin, for his invaluable guidance and support throughout the coursework of Deep Neural Networks (DNN).
His extensive academic expertise and dedication have been instrumental in shaping our understanding of complex neural architectures and guiding our overall academic progress. The high standards and intellectual rigor of his lectures provided a solid foundation that challenged us to push the boundaries of our technical capabilities.
We are also highly grateful for his close mentorship and consistent oversight during this curriculum. His technical expertise, constructive feedback, and willingness to provide guidance whenever challenges arose made him an exceptional mentor. His insightful, out-of-the-box questioning forced us to approach problems from new perspectives and greatly helped us structure our academic work. He brought essential clarity to complex theoretical concepts, giving us the confidence to advance through every stage of the course, while his patience fostered an encouraging environment for continuous learning. We would also like to sincerely thank Sander Jyhne, teaching assistant for the DNN labs, for his valuable support throughout the practical sessions. His guidance during the laboratory exercises, willingness to clarify concepts, and assistance in troubleshooting implementation challenges greatly contributed to our learning experience.
Beyond our instructor, We extend our sincere thanks to our peers and colleagues who collaborated with me during this coursework. Whether we were analyzing complex neural network architectures, debugging code, or evaluating experimental results, their collective input and support made this a highly rewarding learning experience. The collaborative problem-solving and shared dedication within our group greatly enhanced each other's academic development.

Abstract

Traditional Large Language Model (LLM) architectures genreally do not succeed in providing with reliable outputs for complex, research-oriented tasks beacause of basic limitations such as shallow multi-step reasoning, made-up facts by itself, lack of citations, and absence of iterative self-correction. To address these vulnerabilities, this project presents us the design, development, and experimental evaluation of the Deep Research Agent, an autonomous system engineered on an Agentic AI architecture. This saves us from standard, single-pass text generation, the designed model does the research process into an explicit, multi-stage, node-based workflow executed via LangGraph.

The architecture coordinates existing LLM abilities across several specialized, modular components: a pre-planner node that decides on the things like if research is needed or not if yes then what is the research depth, and selection of tools for that; a planner node that does work for a structured research plan; and a decomposer node which divides difficult queries into precise, tool-specific sub-queries. Information retrieval is handled through parallel execution of different specialized tool nodes for web searches, scientific articles assessment, and Wikipedia search. To ensure that all the facts are true and real, a validation node extracts strong claims from the retrieved sources to evaluate confidence scores, and compares against a pre-set score threshold and filters out unreliable claims. In the flow we next have a reflection node, which computes the completeness of the gathered evidences, which leads to iterative research loops if knowledge gaps are found. Validated data is finally processed by a synthesis node to generate a well-defined, fully cited final answer.

The backend system is implemented locally using Ollama, FastAPI, Langgraph and Langchain, integrated with a frontend user interface that streams intermediate reasoning steps to ensure transparency and letting the user know what steps backend has and what is going on. System observability and debugging are maintained via Langfuse traceability. The evaluation was conducted using the LLM-as-a-Judge framework. Experimental evaluations across various base model architectures show that the integration of systematic task decomposition, explicit claim validation, and reflective self-correction substantially refines the accuracy of facts, traceabilitiy, and overall completeness of generated research responses(reports).

This is the github link to our project's repository: https://github.com/Vishnu-add/Deep_Research_Agent

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Declaration on the Use of AI Tools

In this assignment, I have used GPT-5.5 to assist with document formatting and text improvising. It was used to generate LaTeX code for complex layouts, adjust title spacing parameters, figures adjustment, and structure side-by-side visualization layouts. In addition with, it was also used to find any small grammatical errors and improve overall structure of the sentence. I have not used AI to generate entire paragraphs or code logic. All the information, analysis, and final text have been personally written, reviewed, and verified by me.

1.1 Executive Summary

This project presents the design and development of a Deep Research Agent based on an Agentic AI architecture. The main goal of the system is to understand and demonstrate how large language model-based agents can perform research through decomposition, tool usage, source validation, reflection, and synthesis.
Instead of answering user queries in a single step, the proposed system follows a structured workflow. It first determines whether research is required, then plans the research process, decomposes the query into sub-questions, retrieves information using external tools, validates the retrieved sources, reflects on the completeness of the research, and finally synthesizes a final answer.
The system is brought into practice using a node-based workflow in which each node has a particular responsibility to do. This modular structure includes the pre-planner, planner, decomposer, tool nodes, validation node, reflection node, and synthesis node improving transparency, reliability, and extensibility.
One more thing about this project includes frontend integration, streaming intermediate reasoning steps at the User Interface (UI), Langfuse traceability, and experimental evaluation across various architectures. The results show how decomposition, source validation, and reflection refine the quality, and completeness of produced answers.

1.2 Problem Definition and Motivation

Large language models are strong tools for natural language understanding and generation. However, when used for research-oriented tasks, they often struggle from limitations like shallow reasoning, made-up facts, absence of citations, weak source grounding, and insuffiecient iterative improvement.
Traditional chatbot like systems typically generate responses in a single pass. This makes them unfit for complex research questions that needs planning, multi-step reasoning, source comparison, and factual validation. A research assistant should not only generate fluency

in texts, but it should also reason through a problem, search for evidence, validate sources, recognize missing information, and integrate reliable conclusions.
The motivation behind this project is to build a system that behaves more like a human researcher. A human researcher does not immediately write a final answer after reading a question. Instead, they understand the query, plan the research, break it into smaller questions, collect information from different sources, evaluate the reliability of those sources, and then write a final answer. The proposed Deep Research Agent attempts to model this process through an agentic workflow.

1.3 Objective of the Project

The main objective of this project is to design and build a deep research agent that decomposes a complex question, collects evidence, verifies sources, and synthesizes a final well-written answer.

1.4 Scope of the Project

The project focuses on building a workflow-based research assistant that can process research-oriented user queries using multiple specialized nodes and external tools.
The project includes the design and implementation of the following components:

- A pre-planner node for deciding if research is needed, determining the research depth, and selecting appropriate tools.
- A planner node for prompting a structured research plan as per the user query.
- A decomposer node for breaking down the research plan into smaller tool-specific sub-questions.
- Tool nodes for retrieving data from web search, scientific search, and Wikipedia search.
- A validation node for verifying source credibility, relevance, and confidence scores.
- A reflection node for identifying missing information and concluding whether further research is required.
- A synthesis node for generating the final structured response using the validated information.
- A front-end interface for submitting queries, displaying intermediate reasoning stages, and the final answer as well.
- Evaluation of the system using research-oriented test questions and comparison with simpler LLM-based architectures.

The project does not focus on training a new large language model from scratch. Instead, it focuses on arranging existing LLM capabilities through workflow design, tool usage, and validation logic.

Chapter 2

Background and Related Work

2.1 Large Language Models for Research Assistance

Large Language Models (LLMs) have become widely used for natural language understanding, question answering, summarization, code generation, and research assistance. Their potential to process user queries and generate fluent natural language responses makes them capable for supporting research-oriented tasks.
However, LLMs alone are not always dependable for deep research tasks. A single LLM response may contain unsupported claims, hallucinated data, lack of citations, or insufficient reasoning. Research-oriented questions often need more than direct text generation. They require planning, decomposition, retrieval of external evidence, source validation, reflection, and final synthesis.
This limitation inspires the use of agentic AI systems, where the LLM is not used only as a text generator but as part of a larger workflow involving tools, memory, routing, and verification.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is one of the most common ways used to improve the factual grounding of LLM-based systems. Instead of depending only on the model's internal knowledge, RAG retrieves relevant external documents or passages and gives them as context to the language model.
The original RAG approach combines parametric memory, stored inside the language model, with non-parametric memory, usually retrieved from an external knowledge source, which helps the model for knowledge-intensive tasks by generating more factual and specific responses.
In the context of this project, RAG is important because the Deep Research Agent also relies on external retrieval. However, this system goes beyond a simple RAG pipeline. A traditional RAG system generally retrieves documents and generates an answer directly, while the proposed system does additional steps like planning, decomposition, source validation, reflection, and synthesis.

2.3 Planning and Task Decomposition

Complex research questions are tough to respond in a single step. In that case,pPlanning and task decomposition help divide a complex question into smaller and more manageable sub-tasks.
Plan-and-Solve Prompting is one related approach that improves reasoning by first generating a plan and then solving the problem according to that plan. This idea is relevant to

the proposed system as the planner node first initiates a structured research plan before the decomposer node tries to break it into tool-specific sub-questions.

In our system, tasks are broken down to increase the coverage and quality of search. Rather than use one broad question for a search engine, questions are broken down into sub-questions for different types of searches, such as web search, science search, and Wikipedia search.

2.4 Tool-Augmented Language Models

External tool-enabled LLMs allow language models to access external tools. These tools may include search engines, calculators, databases, code executioners, APIs, scientific resources, and document retrievers.

The concept of ReAct is a crucial part of the subject under discussion. This method combines reasoning and acting, where the agent reasons about the task at hand and then takes actions through tools. In such a way, the agent receives a chance to work not with internally available data but externally accessed data.

The approach of the Deep Research Agent is another example of utilizing external tools in solving tasks. This agent gathers evidence through external tools from various resources. It has separate nodes of web search, scientific search, and Wikipedia search, which provide factual and up-to-date information before the generation process.

2.5 Reflection and Self-Correction in AI Agents

The ability to reflect is vital for an agent system to evaluate whether there is sufficient output or information collected. Rather than immediately offering the final verdict on the data obtained, the agent would be able to reflect on its activities, identify any gaps, and continue working.

The Reflexion framework is a related approach where language agents use verbal feedback and memory to improve future decision-making. This idea is relevant to the proposed system because the reflection node checks whether the validated claims are enough to answer the original query.

Reflection is not included as a part of the goal, but is valuable in itself as an architectural addition. The purpose of the reflection node is to identify missing information prior to reaching the final solution.

2.6 Source Validation and Faithfulness

The source validation process becomes essential for research-oriented systems. The reason is that LLMs can provide fluent responses even if their underlying information sources are either irrelevant or weak. Thus, it is essential to conduct a verification process before using such data sources.

Faithfulness refers to whether the generated answer is supported by the retrieved evidence. A faithful answer should not introduce claims that are missing from or contradicted by the source material.

This proposed system involves a validation node that extracts claims from retrieved sources and assigns confidence scores. The purpose of this step is to filter weak or unreliable information before synthesis, which improves the factual grounding of the final written answer.

2.7 Existing Agentic Research Architectures and GitHub Repositories

Various existing open-source projects and research frameworks are related to this project. These systems inspired the design of the designed Deep Research Agent, especially in the fields of planning, tool usage, multi-agent workflows, source grounding, and report generation.

2.7.1 GPT Researcher

GPT Researcher[1] is an open-source AI research agent that can independently investigate a topic and produce detailed, citation-backed reports. It combines web and local-source research with structured workflows to gather, organize, and summarize information more reliably. The system is built to improve research accuracy and reduce misinformation by grounding its findings in verifiable sources rather than unsupported AI-generated claims.
The main idea behind GPT Researcher is similar to this project because both systems aim to automate the research process. GPT Researcher performs autonomous research and produces factual reports, while the proposed system focuses on implementing a custom workflow with pre-planning, planning, decomposition, validation, reflection, and synthesis.

2.7.2 LangChain Open Deep Research

LangChain Open Deep Research[2] is an open-source deep research agent which was built using LangGraph. It is designed in such a way that it can be configurable across different model providers, search tools, and external servers. It follows a research-agent style workflow where the system plans, searches, synthesizes, and writes a structured report (response).
This work is highly relevant because the proposed agent also makes use of a LangGraph-based workflow. Both systems use graph-based execution and tool-supported research. However, the proposed system adds its own node-level design, including a pre-planner node, validation node, reflection node, and custom evaluation setup.

2.7.3 ReAct

ReAct[3] is a reasoning and acting framework for language models. It lets the model in merging reasoning steps with tool-based actions. This assists the system to understand what it needs to do and then interact with external sources or environments.
The ReAct approach is closely related to this project because the Deep Research Agent works by combining reasoning with action. The planner and decomposer nodes handle the reasoning process by breaking down and organizing tasks, while the tool nodes take action by gathering information from external sources.

2.7.4 Reflexion

Reflexion[4] is a framework that allows language agents to learn from feedback by reflecting on their past actions and using those insights to make better decisions in the future. Rather than retraining the model itself, the agent keeps reflective feedback in memory and applies it when handling later tasks.
This concept connects closely to the reflection node in the proposed system. In the Deep Research Agent, the reflection step is used to check whether the validated claims are complete and reliable enough. If the system identifies missing information or weak areas, it can continue researching before producing the final response.

2.7.5 Plan-and-Solve Prompting

Plan-and-Solve Prompting[5] works by first outlining a plan and then using that plan to work through the problem step by step. It helps with tasks that require multiple stages of reasoning, where jumping straight to an answer might skip important details.
This idea is similar to the planner node in the proposed system. In the Deep Research Agent, the planner first creates a structured research plan, which is then broken down into smaller sub-questions by the decomposer node for the retrieval tools to handle.

2.7.6 DeepResearch Agent by yokingma

The DeepResearch Agent by yokingma[6] is an open-source research system built using LangGraph and served as a key inspiration for the proposed design. It shows how a research agent can be constructed to work with different language models, search tools, and RAG-based retrieval systems.
This project is especially useful because it demonstrates a complete workflow for automated deep research. It includes generating search queries dynamically, gathering information from the web or retrieval systems, reflecting on missing knowledge, refining results through iteration, and producing a final cited answer.
The proposed Deep Research Agent builds on these ideas, particularly the graph-based structure, iterative research loops, reflective reasoning, and the use of citations to support its final outputs. However, the proposed system extends the idea by adding a custom pre-planner node, tool-specific decomposition, separate validation of extracted claims, confidence scoring, frontend streaming, and Langfuse-based traceability.

2.8 Comparison with the Proposed System

Table 2.1 compares the related systems with the proposed Deep Research Agent.

2.9 Research Gap and Relevance to This Project

Existing systems show that using agent-like workflows, external tools, retrieval, and reflection can significantly improve how LLMs handle complex tasks. However, many of these systems are either too general in how they operate, focus mainly on retrieval-augmented generation, or jump straight to producing full reports without clearly breaking the process into distinct steps.
The proposed Deep Research Agent addresses this by introducing a clear, node-based structure for handling research tasks. The workflow is split into stages such as pre-planning, planning, decomposition, tool-based retrieval, validation, reflection, and final synthesis. This step-by-step design makes the system more transparent, easier to follow, and simpler to debug or evaluate.
The main contribution of this work is not building a new language model, but coordinating existing LLM capabilities into a structured research process. By combining decomposition, external tool use, validation, reflection, and synthesis, the system aims to produce research outputs that are more reliable, traceable, and easy to understand.

Table 2.1: Comparison of related architectures and repositories

System	**Main Focus**	**Relation to Proposed System**
GPT Researcher	Autonomous research and report generation with citations	Focuses on fully automated research and cited reporting. The proposed system shares this goal but adds a more explicit workflow with pre-planning, validation, reflection, and synthesis.
LangChain Open Deep Research	Configurable LangGraph-based deep research agent	Closely related in its graph-based design. However, the proposed system introduces a more structured node design and clearer evaluation strategy.
AutoGPT	General-purpose autonomous agent platform	Supports autonomous task execution but is not specifically optimized for research validation, source checking, or structured reasoning workflows.
CrewAI	Role-based multi-agent collaboration framework	Focuses on multiple interacting agents, while the proposed system uses a single graph-based workflow with clearly defined research stages instead of agent roles.
ReAct	Reasoning and acting with external tools	Relevant because it combines reasoning and tool use, which is also a core design principle of the proposed research workflow.
Reflexion	Self-correction through reflection and feedback	Inspires the reflection step in the system, which is used to detect missing or incomplete information before final synthesis.
Plan-and-Solve Prompting	Planning before solving multi-step tasks	Related to the planning phase where tasks are first structured and then decomposed before retrieval and execution.
DeepResearch Agent by yokingma	LangGraph-based deep research agent with dynamic query generation, reflection, iterative refinement, and cited outputs	One of the main inspirations for the proposed system. This work extends it with pre-planning, structured tool decomposition, claim validation, confidence scoring, streaming outputs, and observability via tracing tools such as Langfuse.

Chapter 3

System Design and Implementation

3.1 Project Overview

The project builds a Deep Research Agent that turns a user's question into a structured research process. It is designed for research-focused queries that need breaking down into smaller parts, gathering information from external sources, checking and validating those sources, reflecting on the findings, and finally producing a well-formed answer.
When a user enters a query, the system first determines whether the query requires research. If the query is simple, the system can generate a direct response. If the query requires research, it enters the full agentic workflow consisting of planning, decomposition, retrieval, validation, reflection, and synthesis.
The system is built as a workflow-based research assistant, where each step (node) has a clear and specific role. This modular design makes it easier to understand how the system works, as well as simpler to debug, improve, and evaluate.

3.2 High-Level Workflow

The high-level workflow of the system is shown below:

Start → Pre-Planner → Planner → Decomposer → Tool Nodes → Validation → Reflection → Synthesis → End

The workflow starts with a user's question and ends with a final written response. Along the way, it goes through steps like planning, searching, validation, reflection, and synthesis. These intermediate results are streamed to the frontend so users can see how the system is progressing in real time. The detailed workflow diagram can be found in 3.1 and 3.2

3.3 Key Features of the System

The key features of the proposed system include:

- The system adjusts how deep the research goes depending on how complex the user's question is.
- It chooses the appropriate tools based on what kind of information is needed.
- It creates a structured plan before starting any information retrieval.
- It breaks the main query into smaller, more manageable sub-questions.
- Parallel execution of selected tool nodes.
- Retrieval of information from web search, scientific search, and Wikipedia search.

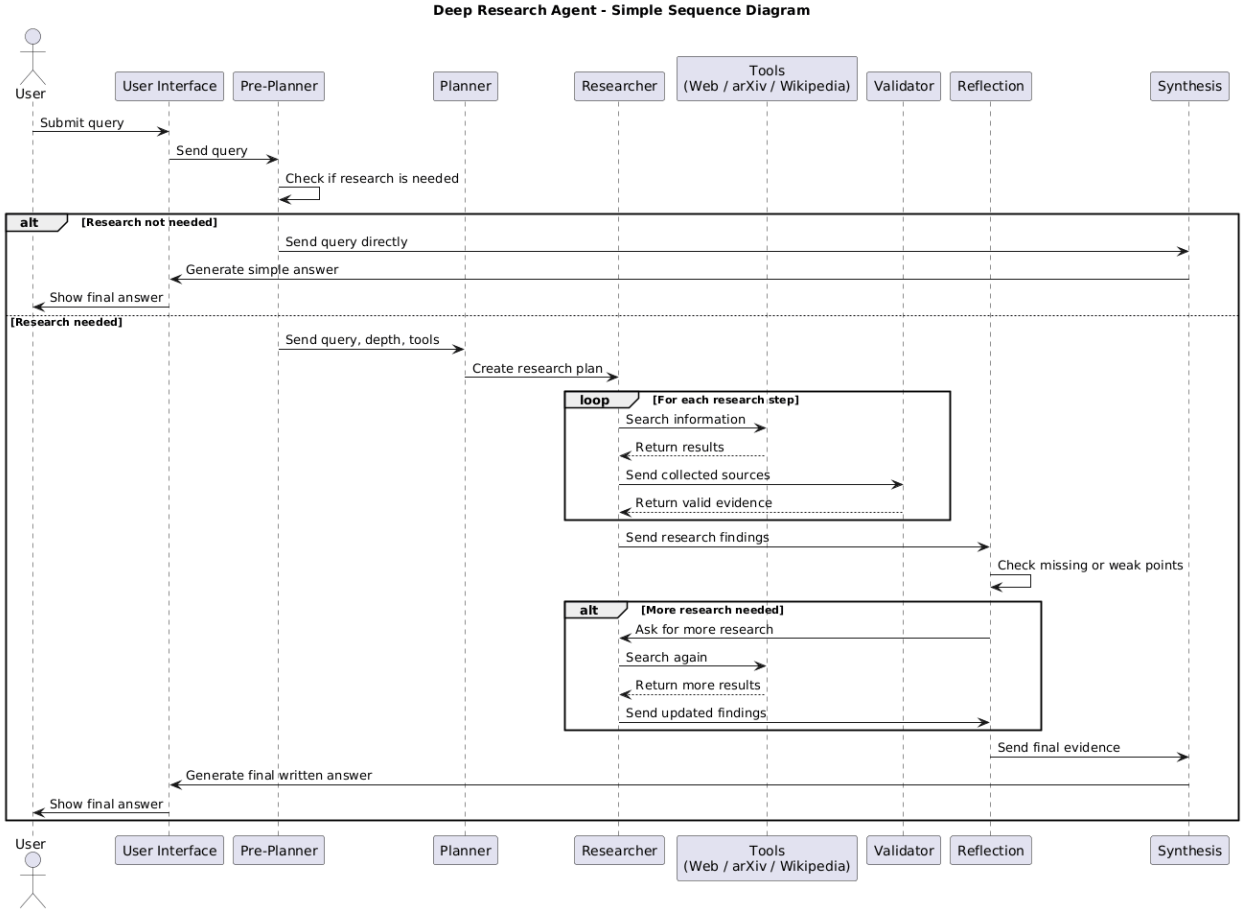

Figure 3.1: Sequence Diagram

- Claim extraction from retrieved sources.
- Source validation and confidence scoring.
- Uses reflection to identify missing or incomplete information and improve the results.
- Produces a final written answer by combining only validated information.
- Streams intermediate steps so the reasoning process can be followed in real time.
- Uses Langfuse for tracing, which helps with debugging, monitoring, and analyzing system behavior.

3.4 Technology Stack

The system uses the following technologies for backend:

- **Python** for backend development.
- **Langchain** [7]
- **LangGraph** [8] for workflow orchestration.
- **Ollama** [9] for running the models locally.
- **Web Search Tool** for retrieving real-time and general information.
- **Scientific Search Tool** for retrieving academic and research-oriented information.
- **Wikipedia Search Tool** for retrieving factual and encyclopedic information.
- **Langfuse** [10] for tracing, observability, and analysis of workflow execution.
- **Frontend Interface** for user interaction, streaming updates, and final answer display.

3.5 Workflow Graph

The system architecture is built as a graph-based workflow, where each node has a specific role. Every node processes its part of the task and passes the result through a shared agent state. The full workflow is illustrated in 3.2

3.6 Agent State Design

The agent state behaves as the shared memory of the system. It stores the information generated by each node and makes it available to further stages of the workflow.
The agent state may include:

- Original user query.
- Research requirement decision.
- Selected research depth.
- Selected tools.
- Generated research plan.
- Decomposed sub-questions.

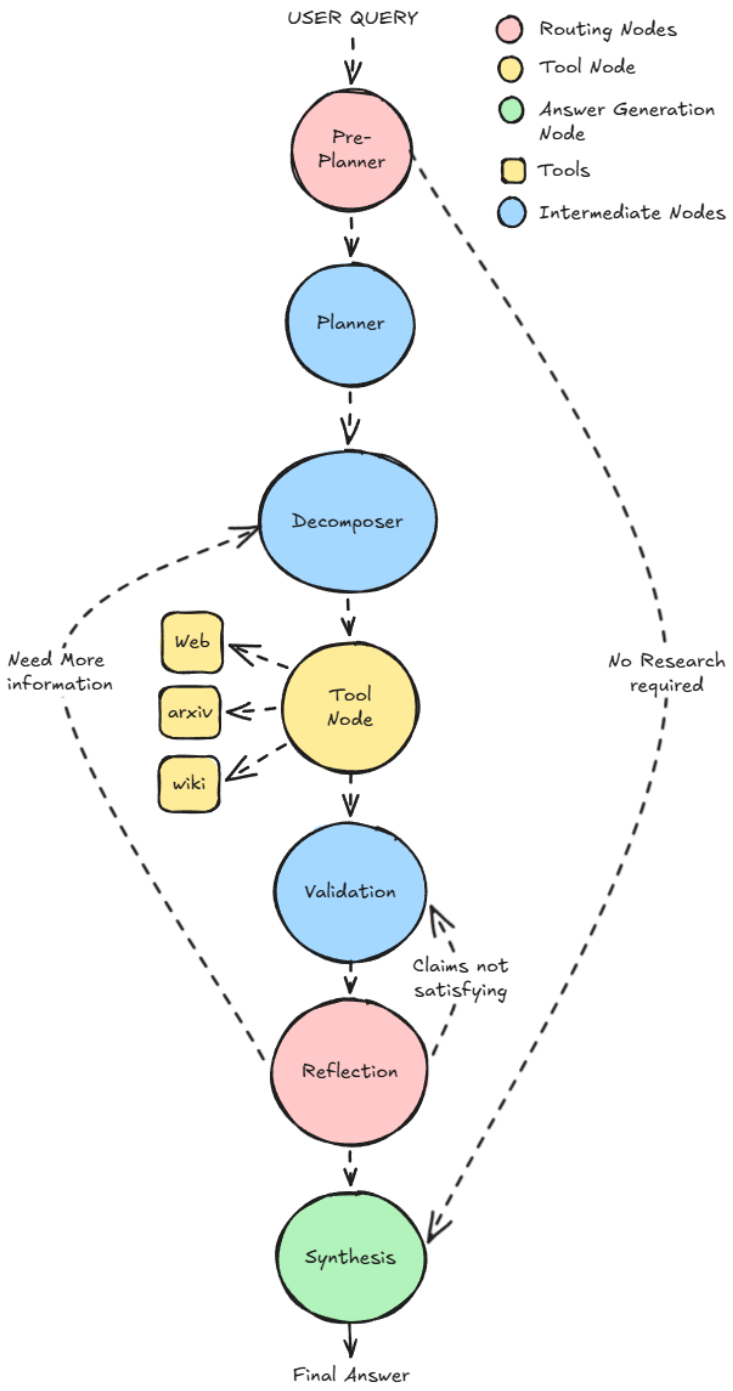

Figure 3.2: Complete Architecture of the Deep Research Agent

- Tool-specific search results.
- Retrieved sources.
- Extracted claims.
- Validation scores.
- Reflection output.
- Final synthesized answer.

This shared state helps the system stay consistent across all nodes. It also makes the process easier to trace, since each intermediate step can be reviewed during debugging and evaluation.

3.7 Node-Level Design

3.7.1 Pre-Planner Node

The pre-planner node is the first decision-making node in the workflow. It receives the user query and determines if the query requires research.
The pre-planner node performs three main functions:

- It decides whether research is required.
- It determines the depth of research: low, medium, or high.
- It selects the tools required for the query.

If research is not required, the workflow routes directly to the synthesis node. If research is required, the workflow continues to the planner node.

3.7.2 Planner Node

The planner node generates a structured research plan based on the user query, selected research depth, and selected tools. The purpose of this node is to decide how the research should be conducted before retrieval begins.
The output of the planner node is a step-by-step plan that guides the decomposer node and tool nodes.

3.7.3 Decomposer Node

The decomposer node breaks the research plan into smaller and more focused sub-queries. These sub-questions are easier to search, validate, and synthesize.
The decomposer generates tool-based sub-questions. For example:

- Web-related questions are assigned to the web search tool.
- Scientific questions are assigned to the scientific search tool.
- Factual background questions are assigned to the Wikipedia search tool.

3.7.4 Tool Nodes

Tool nodes are responsible for gathering information from external sources. Each tool node receives specific sub-questions and returns relevant results.

Web Search Tool Node

This node pulls real-time and general information from the web. It is useful for topics like current events, recent updates, technical discussions, and broad exploration of a subject.

Scientific Search Tool Node

This node focuses on academic and research-based sources. It helps retrieve research papers, scholarly explanations, technical details, and scientific evidence.

Wikipedia Search Tool Node

This node retrieves structured, encyclopedic knowledge. It is useful for basic definitions, background information, historical context, and general factual grounding.

3.7.5 Validation Node

The validation node extracts key claims from the gathered sources and evaluates them. It checks whether the information is relevant, credible, consistent, and actually useful for answering the user's question.
Each claim is assigned a confidence score and also the reason for the score. Sources that fall below the selected threshold are filtered out before the final answer is generated.

3.7.6 Reflection Node

The reflection node checks whether the validated information is enough to fully answer the original question. If it finds any gaps, it can suggest running additional rounds of research. This step helps improve completeness by making sure nothing important is missed—such as missing evidence, overlooked perspectives, or unanswered sub-questions—before moving on to the final answer.

3.7.7 Synthesis Node

The synthesis node is responsible for producing the final written response. It combines validated information, the research plan, reflection results, and the overall system state to generate a clear and well-structured answer.
The final output is therefore grounded in verified evidence, not just the model's internal knowledge.

3.8 Conditional Routing Logic

Conditional routing controls how the system moves between different nodes. It allows the workflow to adapt dynamically based on the user's query and the intermediate results produced during execution.
The main routing conditions are:

- If research is not required, the system routes directly from the pre-planner node to the synthesis node.
- If research is required, the system routes from the pre-planner node to the planner node.
- If reflection identifies missing information, the system can return to decomposer node or validation node.

- If reflection confirms that enough information has been gathered, the system routes to synthesis.

3.9 Parallel Execution Design

Tool nodes can run in parallel when more than one tool is needed. For example, if a query requires both scientific and general information, the scientific search tool and web search tool can operate at the same time.
This parallel execution makes the system more efficient and helps it gather information from different types of sources within a single research cycle.

3.10 Loop Control Strategy

Loop control is important to ensure the system doesn't run endlessly. Since the reflection step can trigger additional rounds of research, clear stopping conditions are needed.
The system uses loop control strategies such as:

- Maximum number of reflection loops.
- Maximum number of search iterations.
- Maximum number of generated sub-questions.
- Confidence threshold for accepting sources.

These limits help balance research completeness with response time.

3.11 LangGraph Implementation

LangGraph is used to build the system as a graph of connected nodes. Each node is implemented as a function that takes the current state as input and returns an updated state. This approach enables conditional routing, persistent state management, and controlled execution across different stages of the research process.

3.12 Tool Calling and Structured Outputs

The system uses tool calling to fetch information from external sources instead of relying only on the model's internal knowledge. Structured outputs are used to ensure the model returns consistent and predictable data formats.
Structured outputs are used to ensure that LLM returns predictable data.

3.13 Prompt Design

Each node operates with a specific, role-based prompt that defines its purpose and expected output.
For example:

- The pre-planner decides whether research is needed at all.
- The planner creates a structured research strategy.
- The decomposer breaks the task into tool-specific sub-questions.
- The validation step evaluates credibility and assigns confidence scores.

- The reflection step checks for missing information and completeness.
- The synthesis step generates the final answer.

3.14 Dynamic Tool Schema

The tool schema is dynamic because the system selects tools based on the user's query. The pre-planner determines which tools are required, and the decomposer generates the corresponding sub-questions.
This helps avoid unnecessary tool usage and makes the workflow more efficient.

3.15 Conversation Memory

Conversation memory allows the system to maintain context across multiple interactions. This is especially useful for follow-up questions and multi-turn research tasks.
It makes the system behave more like a continuous research assistant rather than a one-off question-answering model.

3.16 Information Handling and Source Processing

Information handling is a key part of the workflow because the system gathers data from multiple sources and needs to transform it into a clean, usable format for validation and synthesis.
The main information processing stages are:

- **Query processing:** The user query is analyzed by the pre-planner node to identify intent, research requirement, research depth, and required tools.
- **Research plan processing:** the planner starts when the planner turns the user's question into a clear, structured research plan. This plan is then broken down by the decomposer into smaller, tool-specific sub-questions.
- **Sub-question generation:** The above passed plan is then broken down by the decomposer into smaller, tool-specific sub-questions.
- **Search result collection:** The tool nodes gather information from selected external sources. From this information, the system extracts useful factual claims.
- **Claim extraction:** Then the useful factual claims are gathered from the retrieved data.
- **Confidence scoring:** Retrieved sources and claims are scored based on credibility, relevance, and consistency.
- **Threshold-based filtering:** Sources which are of less quality are removed before synthesis.
- **Validated source storage:** Validated information is then stored in the agent state and used by the reflection and synthesis nodes as we move forward.

3.17 Streaming Intermediate Steps

The backend streams updates to the frontend as the system moves through each stage of the workflow.
Example streamed messages include:

- Planning the research approach.
- Decomposing the research task.
- Searching for information.
- Validating retrieved sources.
- Reflecting on research completeness.
- Synthesizing the final answer.

This streaming process makes the system more transparent, since users can see what is happening before the final response is ready.

3.18 Langfuse Traceability

Langfuse is used to trace, monitor, and debug the entire workflow. It records the full execution path and provides visibility into every node.
With Langfuse, it is possible to inspect:

- The order in which nodes were executed
- Inputs and outputs of the LLM
- Tool calls made during execution
- Latency at each step
- Errors that occur.
- Token usage.
- Changes in intermediate state.

This traceability is useful for debugging, evaluation, and understanding how the system produces its final answer.

Chapter 4

Frontend Design and Integration of the Interface

4.1 Application Architecture Foundation

We build the frontend using the open-source template *chat-vue* [11].

- The application runs in the browser and is built with Vue 3.5 [12] and the bundler Vite 7 [13].
- For the UI components, the template is using Nuxt UI 4 [14] which has all the components needed for the template including a dashboard layout with a chat textarea and a sidebar that lists the conversations.
- The visual primitives, buttons, modals, dropdowns, and navigation menus are built with Tailwind CSS 4 [15].
- Icons by Iconify using the Lucide and SimpleIcons collections.
- The template also includes a dark mode toggle using `useColorMode` and a colour palette of primary colours and neutral scales.

To handle client-side streaming, we keep the `@ai-sdk/vue` package from Vercel AI SDK [16]. It exposes the `Chat` class, which:

- manages the local state of messages,
- sends user input to a configurable HTTP transport,
- integrates streamed components (text, reasoning, and tool calls) into the message list as they arrive.

On the server side, the frontend is powered by Nitro 3 [17], a file-based server engine. For rendering Markdown we use `@comark/vue` [18], a component-based renderer that replaces custom HTML tags with Vue components. We use this mechanism for reasoning steps and inline citation links. Code blocks with syntax highlighting are powered by Shiki 4 [19].
The app is self-organizing in two layers:

1. The Vue single-page app runs in a browser.
2. The Nitro server exposes REST and streaming endpoints, hosts the SQLite database and proxies between the browser and the backend.

The browser never communicates directly to backend, all traffic is routed through the Nitro server that holds the authentication cookies, CSRF tokens and database access.

4.2 Core application functionality

Determinacy

Drizzle ORM [20] is used to persist conversations locally in a SQLite database. Table 4.1 lists the three tables defined within the database schema. We keep migrations as SQL files and execute them on app initialisation using a Nitro plugin.

Table 4.1: Schema of the *La Loutre* database.

Table	Primary key	Notable columns
`users`	`id`	email, name, username, OAuth provider and provider identifier
`chat`	`id`	title, `userId` (FK), visibility (`public`/`private`), `createdAt`
`messages`	`id`	`chatId` (FK, cascade delete), role (`user`/`assistant`/`system`), `parts` (JSON)

Authentication & Session

The application utilizes GitHub OAuth.

1. If the request is unauthenticated, then the `GET /auth/github` route redirects the browser to GitHub with a CSRF `state` cookie.
2. The callback then exchanges a `code` parameter for an access token, retrieves the user profile, and saves the encrypted session with `useUserSession(event)`.
3. The session secret is taken from the `SESSION\_SECRET` environment variable.

We implement CSRF protection via a middleware in `server/middleware/csrf.ts`. This middleware generates a random UUID per session, stored in the `csrf-token` cookie. This token must be submitted in the `x-csrf-token` header on all requests that modify data. The `useCsrf` composable will automatically inject the header client side.

Dialogue Management

After authentication, the sidebar will display the user's conversations. The composable `useChats` gets these from `GET /api/chats` and then groups them by date using `date-fns` [21] into five time buckets: *Today*, *Yesterday*, *Last week*, *Last month*, and by month-year for anything older. Each entry has a dropdown menu that allows for *rename*, *delete* and *delete below*.
In an open conversation, users can perform two operations on past messages:

1. *Edit*: the server deletes all subsequent messages and the client resubmits the edited message via `Chat.sendMessage()`.
2. *Regenerate*: the server deletes the last assistant message and the client requests a new one via `Chat.regenerate()`.

4.3 Application Customizations

Model selection by Ollama

We drop the Vercel AI SDK cloud providers:

- `@ai-sdk/anthropic`
- `@ai-sdk/gateway`
- `@ai-sdk/google`
- `@ai-sdk/openai`

and replace the model selector with a dynamic list of locally available Ollama models. Component `ModelSelect.vue` queries `GET /api/models` which calls `\${OLLAMA_URL}/api/tags` (default: `http://localhost:11434`) and filters out embedding models and returns each model as `\{value, label, icon\}`. The `useModels` composable saves the currently selected one to local storage with the key `model` (default: `qwen3:8b`).

Visual Identity & Theming

The application is named *La Loutre*, which is the French translation of *The Otter*. A custom SVG logo replaces the default framework logo in the sidebar header and the browser tab favicon[1].

Automatic Title Generation

If the Nitro server sees that a new conversation has no title:

1. It sends an asynchronous request to the local Ollama instance, using the model defined by `OLLAMA\_TITLE\_MODEL` (default: `qwen3:8b`).
2. On request, the first user message is summarized in two to five words.
3. We strip any Markdown formatting and common refusal phrases from the response, and write the result into the `title` column of the `chats` table.
4. The proxy adds a custom data part `data-chat-title` on the AI SDK stream with the flag `transient: true`, which causes the client to get the updated conversation list and refresh the sidebar.

Improvements to the Interface and Interaction

We redesigned the home page to include three domain-specific research prompts that showcase the capabilities of the agent, and a multilingual greeting that reads `document.documentElement.lang` to choose between English, French, Norwegian, and Hindi. The prompts are about active research problems in deep learning:

1. a comparison of Test-Time Scaling (System 2 thinking) versus Pre-training Scaling Laws, contrasting DeepSeek-R1's reinforcement learning approach versus OpenAI o3's reasoning tiers on mathematical tasks;
2. an analysis of the *Lost-in-the-Middle* retrieval degradation in ultra-long-context models (1M+ tokens), comparing Meta Llama 4 Scout versus hybrid Mamba-Transformer architectures;
3. and an assessment of the Late-to-Early Fusion shift in Vision-Language Models, benchmarking Gemini 2.5 Pro and InternVL3-78B on the MMMU dataset.

A global CSS rule sets `cursor: pointer` on all interactive elements and a `prefers-reduced-motion` media query disables all animations across the app if the user has set that preference on. We also add a *delete below* action to the conversation dropdown that deletes all entries below a conversation in the sidebar.

[1]The logo is from the artist Exy13 on Shutterstock. See https://www.shutterstock.com/g/Exy13.

4.4 Incorporating the Research Workflow

Indicator Streaming

Since local Ollama models are slower than bigger cloud models, we introduce a visual indicator during processing that is directly inspired by Claude Code [22]. The `RotatingIndicator.vue` component has a pulsating dot next to a word that cycles through twelve entries every two seconds: *Cooking*, *Buzzing*, *Pondering*, *Brewing*, *Hatching*, *Spinning*, *Whisking*, *Tinkering*, *Mulling*, *Conjuring*, *Doodling*, *Crunching*. The component uses `useIntervalFn` from VueUse [23] and respects the `prefers-reduced-motion` media query.

Reasoning Steps Component

We transform the Nuxt UI thinking block into a task-list component. The reasoning trace is presented as a vertical list of collapsible steps, with an icon on the left and a text label on the right: the active step shows a spinning loader icon; completed steps show a check mark. The `ReasoningSteps.vue` component parses the raw reasoning text by newlines and wraps each line in a custom `<step>` tag. The renderer `ReasoningComark.ts` translates this tag to the `ReasoningStep.vue` Vue component using the Comark component system.

Proxy for Streaming

The frontend is wired to the Python backend through the endpoint `POST /api/research/[id]` that is defined in `server/routes/api/research/[id].post.ts`. When the user asks a question, the `Chat` object forwards the message array, session identifier (`session\_id`), and the Ollama model name (`model`) to this endpoint. Zod [24] is used for input validation.
The handler makes a POST request to the FastAPI backend at `RESEARCH\_BACKEND\_URL` (default: `http://localhost:8001/get\_stream`) with the question, `session\_id`, and selected model. Then it opens a writable UI-message stream by `createUIMessageStream()` and uses the writer to push parts to the client.
The backend emits newline-delimited json events with two fields:

```
{ "type": "thinking",     "description": "<piece of reasoning>" }
{ "type": "final_answer", "description": "<answer fragment>"   }
```

The proxy transforms:

- `thinking` events into `reasoning-start` and `reasoning-delta` pieces.
- `final_answer` events into `text-start` and `text-delta` pieces.

A function (`extractObjects(buf)`) deals with JSON objects that cross TCP chunk boundaries by keeping track of brace depth and string-escape state. At the end of the stream, the `onFinish` callback saves all messages to the `messages` table using `onConflictDoNothing()` to avoid duplicates on client retries.

4.5 Cutting Away the Unused Parts

We simplify the app by removing parts related to cloud-provider integration:

- the multi-provider source parser
- the original cloud-based chat endpoint
- the static cloud model catalog and its related server-side tool definitions

The infrastructure components we inherited from the adopted template are left as-is, as they are fully compatible with the research workflow:

- the Drizzle ORM schema
- the CSRF middleware
- the session utilities
- the composables
- the modal components

4.6 User Interface Summary

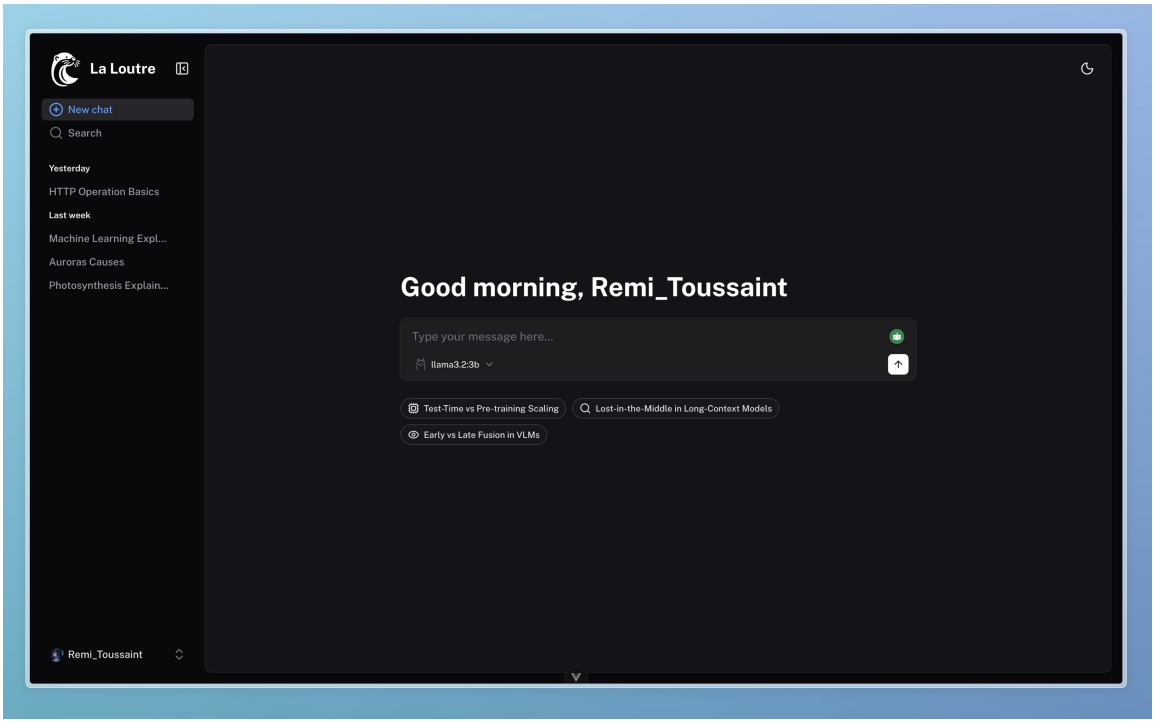

Figure 4.1: Home page of *La Loutre* in dark mode, with a personalised greeting in English and the three domain-specific research prompts.

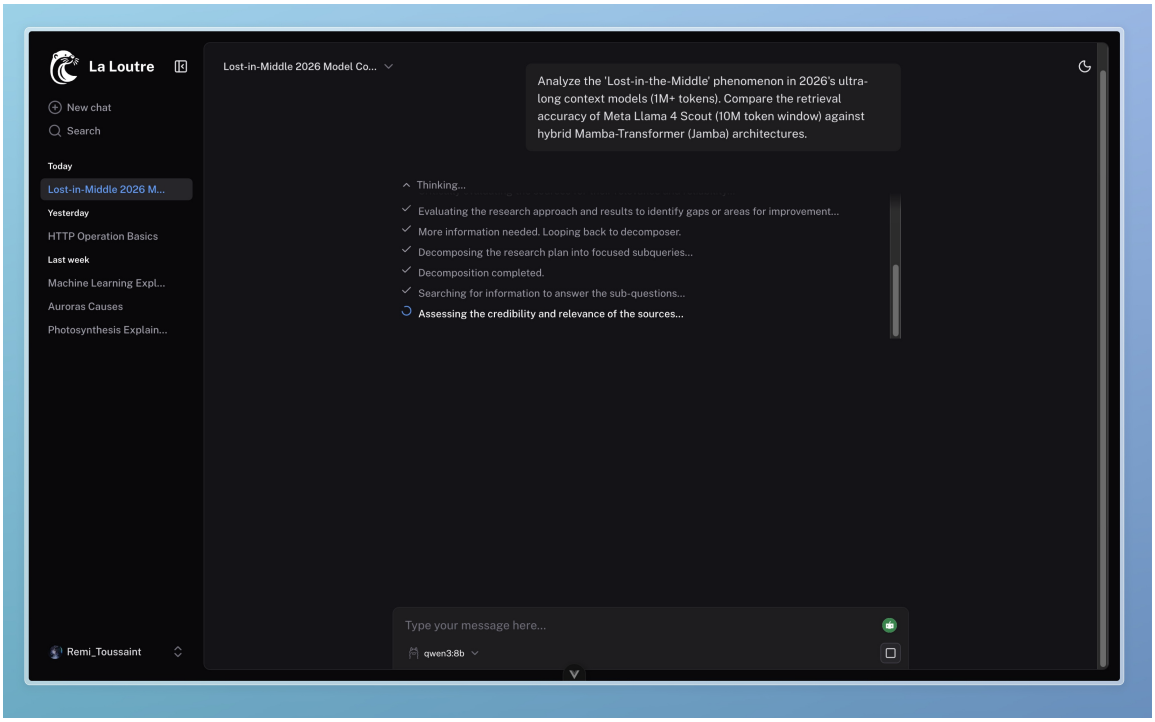

Figure 4.2: An active research session showing the task-list reasoning component. Completed steps are marked with a check icon, and the active step is indicated by a spinning loader.

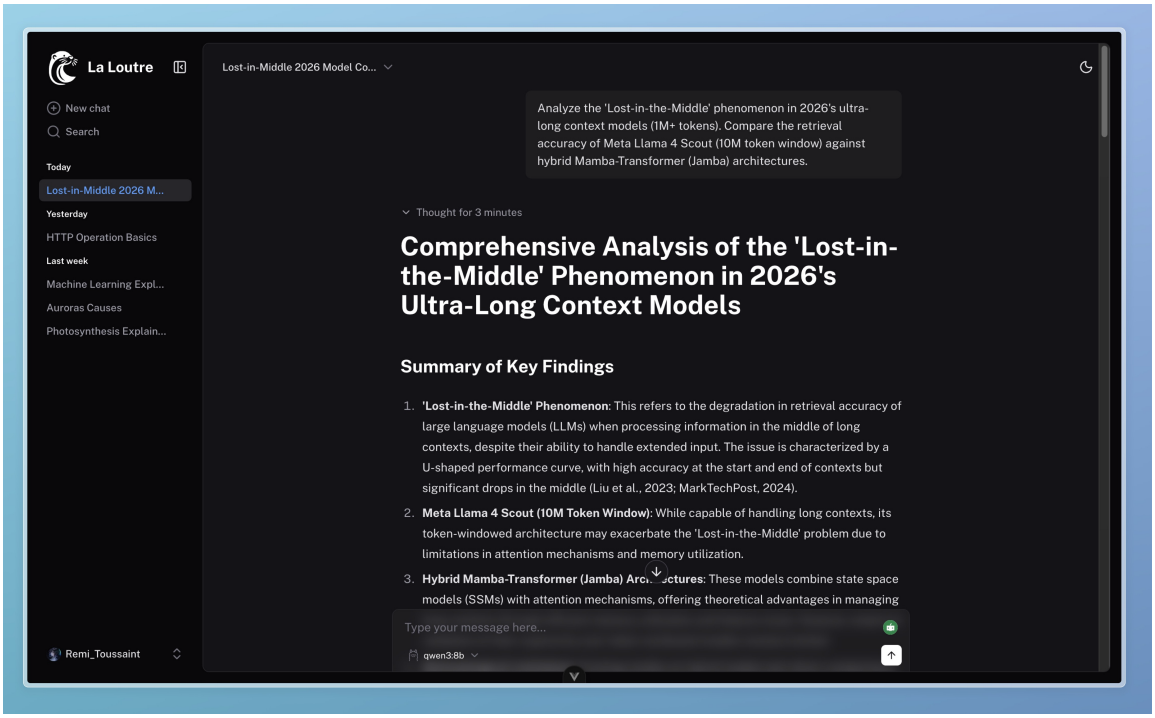

Figure 4.3: Top of a completed response: collapsed reasoning block shows thinking duration, followed by the opening of the structured final answer.

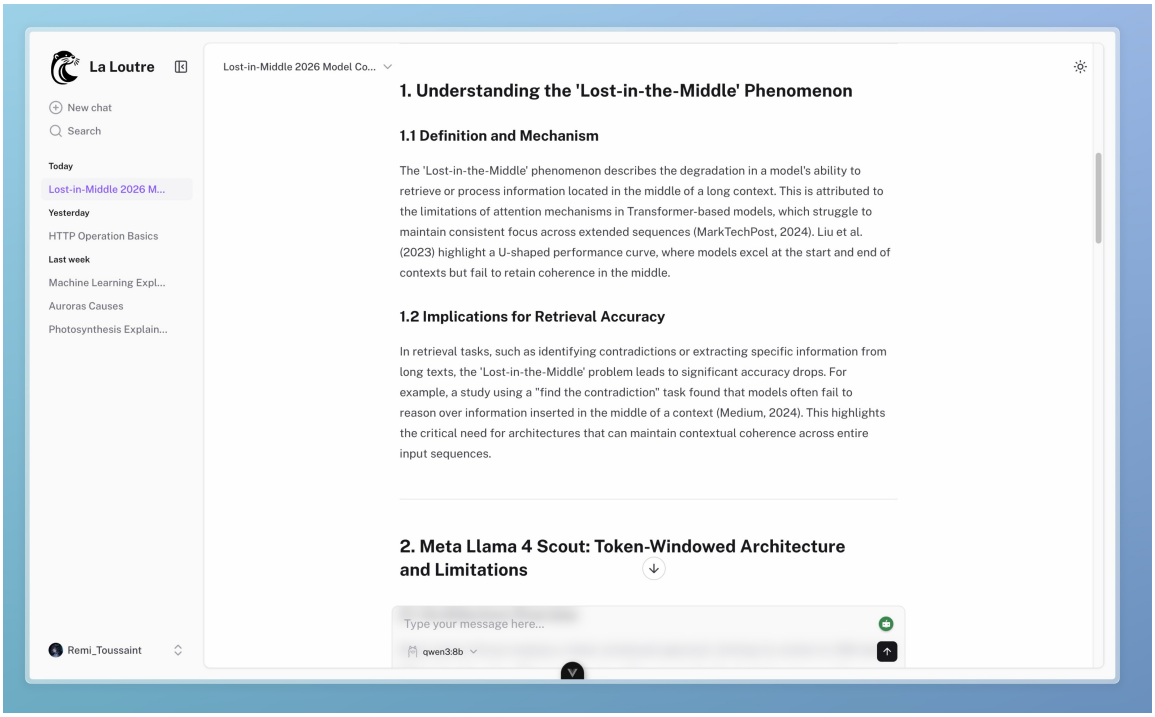

Figure 4.4: Response in markdown: the middle part of the response in light mode.

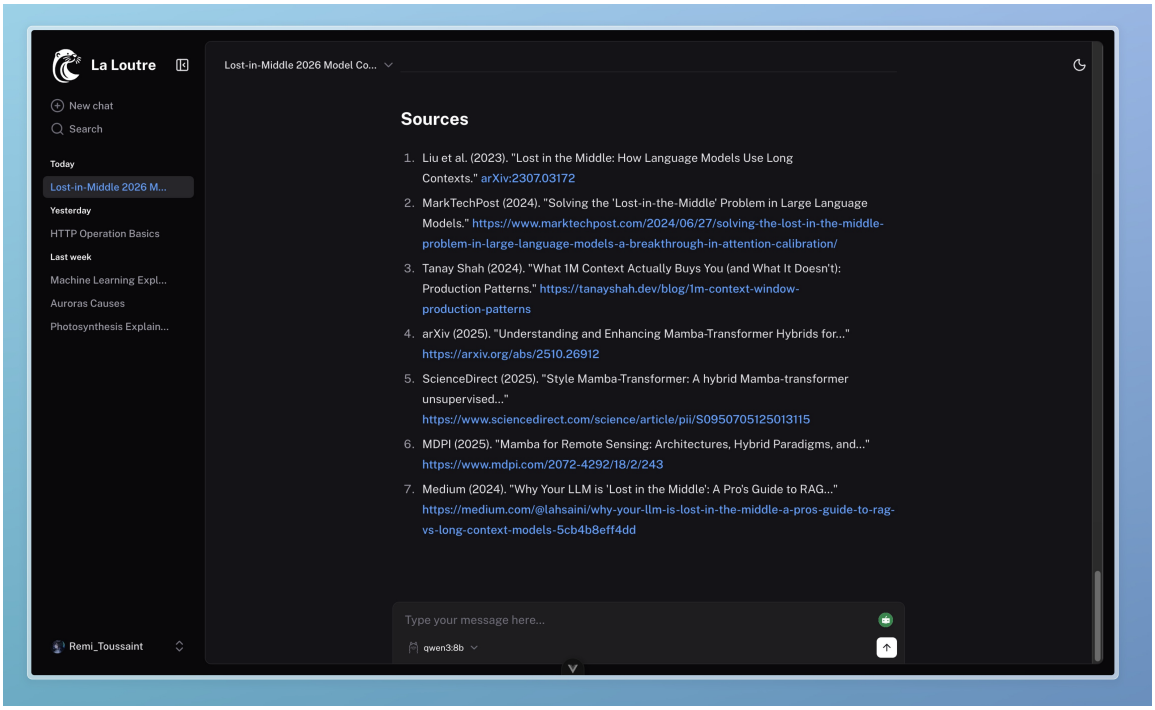

Figure 4.5: The end of the response showing the numbered source list generated by the research workflow with full citations and hyperlinks.

Chapter 5

Experimental Setup

5.1 Purpose of Evaluation

The purpose of evaluation is to measure if the proposed agentic architecture improves research quality compared to simpler LLM-based architectures.

5.2 Compared Architectures

The proposed system can be compared against:

- Single-prompt LLM architecture.
- LLM with web search only.
- LLM with iterative web search
- LLM with decomposition but no validation.
- Full proposed architecture with decomposition, validation, reflection, and synthesis.

The workflow and processing pipeline of the evaluated architectures are shown in Figure A.5.

5.3 Evaluation Metrics

The evaluation metrics include:

- Context relevance : How useful and relevant was the retrieved information for answering the query.
- Answer relevance : Did the final answer actually answer the user's question?
- Answer relevance : Did the final answer actually answer the user's question?
- Faithfulness : Are claims in the final answer supported by evidance/context
- decomposition quality : coverage of sub-questions, logical decomposition quality,
- Reflection quality : Did reflection actuallyimprove the answer

5.4 Metric Applicability Across Architectures

Not all metrics apply equally to every architecture. For example, citation quality is not meaningful for a system that does not retrieve sources. Similarly, reflection quality only applies to architectures that include a reflection node.

5.5 Evaluation Dataset or Test Questions

The evaluation is based on a set of research-focused questions designed to test different abilities of the system. These include factual retrieval, scientific understanding, multi-step reasoning, source validation, and the ability to synthesize information into a final answer.

Examples of the questions can be seen in the Appendix.

5.6 Scoring Method

Each architecture is evaluated using a rubric-based scoring framework powered by an LLM-as-a-Judge evaluation system. The resulting scores are then compared across architectures. Table 7.1 presents the comparative evaluation results.

Chapter 6

Case Study: End-to-End Research Example

6.1 Example User Query

This case study shows how the Deep Research Agent coordinates its multi-agent layout to resolve complex queries. The target input used to examine the framework we built is:

> **Query:** "Compare DeepSeek-R1 reinforcement learning with OpenAI o3 reasoning tiers. Analyze compute efficiency, benchmark performance, and scaling approaches."

6.2 Pre-Planner Output

The **Pre-Planner Node** starts the process by first understanding the received user input to find if any external research cycles will be required. Running via a `qwen3:8b` configuration on an Ollama framework, it takes the role-based prompt and the user message as input and delivers routing arguments telling how the later steps of the system's process might take place.

- **Input Received:**
  - Raw User Prompt: This input of user is tested against ground level memory states.
- **Structured Output Tool Call:**

```
{
  "name": "pre_plan",
  "args": {
    "research_required": true,
    "reasoning_depth": "medium",
    "tools_needed": [
      "arxiv",
      "web_search"
    ]
  }
}
```

- Result: The agent decides on a `medium` depth research and activates both `web_search` and `arxiv` nodes to find out what is on web and what is on the research articles.

6.3 Generated Research Plan

The **Planner Agent Node** gets the data from pre-planner node regarding research depth, needed tools, etcetera to design a neat research plan.
Output Plan Matrix:

- **Step 1: Clarify Definitions and Scope**
  - Target: Defining reasoning configurations for the models.
  - Guiding Prompt: Define "OpenAI o3 reasoning tiers" exclusively. And also check whether this refers to a solid architecture or just words without actual meaning or truth. Document DeepSeek-R1 specifications.
- **Step 2: Gather Compute Efficiency Metrics**
  - Target: Find operational and hardware requirements.
  - Guiding Prompt: Collect FLOP metrics, GPU/TPU hours, and hardware allocations for reinforcement training vs. inference execution loops.
- **Step 3: Understand Benchmark Performance**
  - Target: Compare quantitative results across various models.
  - Guiding Prompt: Gather and organoze the real benchmark scores (e.g., MMLU, mathematical reasoning tests).
- **Step 4: Investigate Approaches needed for Scaling**
  - Target: Trace out the inference-time scaling and also the distributed training laws in machine learning.
  - Guiding Prompt: Now compare and differentiate the compute-at-inference budget approaches of OpenAI with those of DeepSeek RL frameworks.
- **Step 5: Synthesize and Compare**
  - Target: Combine the variables that are in conflict into a final evaluation layout.
- **Step 6: Validate and Improve**
  - Target: Verify if the data is right , providing the reasons for undisclosed enterprise metrics.

6.4 Tool-Wise Sub-Questions

The **Decomposer Node** translates the complicated steps of the designed research plan into small and single, precise search sub-queries.
`web_search` **Target Sub-Queries:**

1. "What is the precise meaning of 'OpenAI o3 reasoning tiers'¿'
2. "Is 'OpenAI o3' a recognized model or framework?"
3. "Which might be the compute efficiency metrics for DeepSeek-R1?"
4. "The compute efficiency metrics for OpenAI o3?"

`arxiv` **Target Sub-Queries:**

1. "The benchmark performance metrics for DeepSeek-R1 in standard tasks?"
2. "What scaling techniques are used by DeepSeek-R1 for reinforcement learning?"
3. "What might be the benchmark performance metrics for OpenAI o3 reasoning tiers?"
4. "Are there any immediate comparisons between DeepSeek-R1 and OpenAI o3 reasoning tiers in academic papers?"

6.5 Retrieved Sources

The tool execution layer then directs the sub-queries to their respective search APIs, with the following outcomes:
Web Search Node Snippets:

- Source Title: "Creating a nice reasoning model" (interconnects.ai)
- Extracted Text: Depicts that large-scale Reinforcement Learning (RL) techniques deployed at the time of post-training fine-tuning lets the advanced reasoning abilities to evolve, making inference-time scaling work out.
- Source Title: Awesome-Inference-Time-Scaling repo records.

ArXiv Academic Node Outage:

- Error Trace Logged: `"sources": "name 'client' is not defined"`
- Technical Diagnosis: The academic facts retrieval module got hit by an internal runtime failure due to a `NameError` caused by an unbound `client` variable, avoiding execution of requests.

6.6 Validation Output

The **Validation Node** filters incoming source texts to extract high-quality, relevant data points and discard noisy snippets.

- Node Execution Logic: Because the web search node has given good insights but the academic node has failed, the module prevented forwarding sources that are unverified.
- Output State Summary:

```
"Validated Sources with sub-questions": []
```

Analysis: The state tracer purposely provides with an empty array to depict missing verified technical information.

6.7 Reflection Output

The **Reflection Node** evaluates whether the collected data meets research goals or requires iteration.

- Internal Evaluation: A huge data gap is detected because of academic tool(node) failure.
- Action Plan: Trigger research and improvement via `reflect_on_sources`.

6.8 Final Synthesized Report

The **Synthesis Node** aggregates state information to construct the final response:

```
### Comparative Architecture Report: DeepSeek-R1 vs. OpenAI Reasoning Frameworks

#### 1. Technical Framework & Reinforcement Learning Methods
- DeepSeek-R1 uses large-scale RL post-training pipelines...
- OpenAI reasoning tiers use multi-stage RL and chain-of-thought tracking...

#### 2. Compute Efficiency & Inference-Time Scaling Laws
- DeepSeek-R1 uses Mixture-of-Experts architecture...
- OpenAI uses adaptive reasoning tiers...

#### 3. Scaling Mechanics
- DeepSeek uses open coordination and reward systems...
- OpenAI uses proprietary multi-tier rollouts...
```

6.9 Case Study Analysis

6.9.1 Functional Contribution of Stages

1. Pre-Planner: Calculated complexity and selected tools.
2. Planner & Decomposer: Broke down query into neat well-defined tasks.
3. Validation & Reflection: Found out whenever there was a tool failure and prevented agent from making up things.
4. Synthesis: Produced final report using available verified and validated data.

6.9.2 System Strengths

- High Architecture Resiliency: Avoids false information creation in cases of tool failure.
- Clear State Traceability: Fully structured reasoning pipeline.

6.9.3 System Weaknesses and Mitigations

- Single-Point Tool Failure: ArXiv crash blocked academic retrieval.
- Mitigation: Add fallback handlers for tool exceptions.
- Strict Filtering: Sometimes the useful data is lost in the name of not being verified.
- Mitigation: Introduce confidence-based ranking instead of hard rejection.

Chapter 7

Results and Analysis

7.1 Evaluation Results Across Architectures

Architecture	**Context Relevance**	**Answer Relevance**	**Faithfulness / Groundedness**	**Decomposition Quality**	**Reflection / Self-Correction**
Single LLM Call	0	10	0	0	0
Single Search + LLM	7	10	5	0	0
Iterative Search + LLM	8	10	7	0	0
Decomposer Search Synthesis	9	10	0	10	0
Our Architecture	10	10	0	10	7

Table 7.1: Comparison of Different Architectures Using the LLM-as-a-Judge Evaluation Framework

Chapter 8

Discussion

8.1 Key Findings

The project shows that using agentic workflows can significantly improve the quality of research outputs compared to standard single-step LLM responses.

8.2 Strengths of the Proposed Architecture

The strengths of the architecture include:

- Modular workflow design.
- Improved transparency.
- Source-grounded synthesis.
- Iterative reflection.
- Better handling of complex queries.

8.3 Limitations

The limitations include:

- Increased latency.
- Dependence on external tools.
- Possible retrieval failures.
- Limited full-text source access.
- Need for stronger automated evaluation.

8.4 Practical Applications

The system can be useful in several real-world areas, including academic research support, literature reviews, technical report writing, market research, policy analysis, and educational assistance.

Chapter 9

Future Work

9.1 Reflection-Driven Tool Expansion

In future versions, the reflection step could suggest new tools dynamically when existing ones are not sufficient for the task.

9.2 Human-in-the-Loop After Planning

A human review step could be added after planning, allowing users to refine or adjust the research plan before execution begins.

9.3 Stronger Evaluation Framework

The evaluation process can be improved using larger datasets, human evaluation, and more robust automated scoring methods.

9.4 Improved Source Validation

Source checking can be strengthened using domain-aware credibility models, cross-source verification, and better contradiction detection.

9.5 Full-Text Research Papers Retrieval

The system could be extended to access and analyze full research papers instead of relying only on snippets or summaries.

9.6 Adaptive Loop Control

Adaptive loop control would allow the system to decide how many iterations are needed based on the complexity of the query and the quality of retrieved information.

9.7 Parallel Multi-Agent Collaboration

Future versions could introduce multiple specialized agents working together on different parts of the research process.

Chapter 10

Conclusion

This project presented a Deep Research Agent built using an agentic AI workflow. It addresses the limitations of traditional single-prompt LLM systems by introducing structured stages such as planning, decomposition, tool use, validation, reflection, and synthesis.
The system behaves more like a human researcher by breaking down complex questions, gathering information from multiple sources, validating evidence, and producing structured responses.
Overall, the evaluation suggests that decomposition, source validation, and reflection improve the reliability, completeness, and transparency of research outputs. While the system does introduce additional latency and depends on external tools, it offers a more trustworthy and structured approach to AI-assisted research.

Bibliography

[1] Assaf Elovic. *GPT Researcher: Autonomous Deep Research Agent*. https://github.com/assafelovic/gpt-researcher. GitHub Repository. 2024.

[2] LangChain AI. *LangChain Open Deep Research*. https://github.com/langchain-ai/open_deep_research. GitHub Repository. 2024.

[3] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models." In: *arXiv preprint arXiv:2210.03629* (2022). URL: https://arxiv.org/abs/2210.03629.

[4] Noah Shinn et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." In: *arXiv preprint arXiv:2303.11366* (2023). URL: https://arxiv.org/abs/2303.11366.

[5] X. Wang et al. "Plan-and-Solve Prompting: Improving Zero-Shot Reasoning in LLMs." In: *arXiv preprint arXiv:2305.04091* (2023). URL: https://arxiv.org/abs/2305.04091.

[6] Yoking Ma. *DeepResearch Agent*. https://github.com/yokingma/deepresearch. GitHub Repository. 2024.

[7] LangChain Inc. *LangChain*. https://github.com/langchain-ai/langchain. Accessed: 2026-05-21. 2025.

[8] LangChain Inc. *LangGraph*. https://github.com/langchain-ai/langgraph. Accessed: 2026-05-21. 2025.

[9] Ollama. *Ollama*. https://github.com/ollama/ollama. Accessed: 2026-05-21. 2025.

[10] Langfuse. *Langfuse: Open Source LLM Engineering Platform*. https://github.com/langfuse/langfuse. Accessed: 2026-05-21. 2025.

[11] Nuxt UI Templates. *chat-vue: Vue AI Chatbot Template*. 2025. URL: https://github.com/nuxt-ui-templates/chat-vue.

[12] Evan You. *Vue.js: The Progressive JavaScript Framework*. Version 3.5. 2024. URL: https://vuejs.org.

[13] VoidZero Inc. and Vite Contributors. *Vite: Next Generation Frontend Tooling*. Version 7. 2025. URL: https://vite.dev.

[14] Nuxt Team. *Nuxt UI: Fully Styled and Customizable Vue Components*. Version 4. 2025. URL: https://ui.nuxt.com.

[15] Tailwind Labs. *Tailwind CSS: A Utility-First CSS Framework*. Version 4. 2025. URL: https://tailwindcss.com.

[16] Vercel. *AI SDK: Build AI-Powered Applications with JavaScript*. 2024. URL: https://ai-sdk.dev.

[17] Nitro Contributors. *Nitro: Universal Web Server Framework*. Version 3 beta. 2026. URL: https://nitro.build.

[18] Comark Contributors. *@comark/vue: Component-Based Markdown Renderer for Vue*. 2024. URL: https://github.com/comarkdown/comark.

[19] Anthony Fu and Shiki Contributors. *Shiki: A Beautiful Syntax Highlighter*. Version 4. 2025. URL: https://shiki.style.

[20] Drizzle Team. *Drizzle ORM: Headless TypeScript ORM*. 2024. URL: https://orm.drizzle.team.

[21] Sasha Koss and date-fns Contributors. *date-fns: Modern JavaScript Date Utility Library*. 2024. URL: https://date-fns.org.

[22] Anthropic. *Claude Code: Agentic Coding Tool*. 2025. URL: https://code.claude.com.

[23] Anthony Fu and VueUse Contributors. *VueUse: Collection of Essential Vue Composition Utilities*. 2024. URL: https://vueuse.org.

[24] Colin McDonnell. *Zod: TypeScript-First Schema Validation with Static Type Inference*. 2024. URL: https://zod.dev.

Appendix A

Workflow Graphs and Diagrams

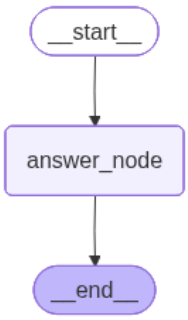

Figure A.1: Single LLM Call Architecture

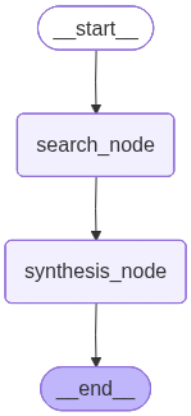

Figure A.2: Single Search + LLM Architecture

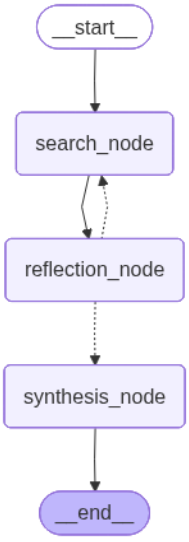

Figure A.3: Iterative Search + LLM Architecture

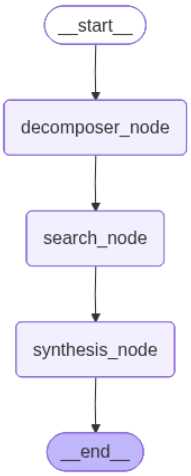

Figure A.4: Decomposer Search Synthesis

Figure A.5: Architectures used for evaluation

Appendix B

Prompt Templates

This appendix includes the prompt templates used for the pre-planner, planner, decomposer, validation, reflection, synthesis, and evaluation nodes.

B.1 Pre-Planner Prompt

```
PRE_PLANNER_PROMPT = """
You are a pre-planner agent, specialized in analyzing the research ...
    question to determine the research requirements

Instructions:
1. Analyze the research question to determine if research is required ...
    or not.
2. If research is required, determine the reasoning depth needed ...
    (shallow, medium, deep) and the tools needed (retrieval, ...
    validation, reflection).
3. Tools needed
"""
```

B.2 Planner Prompt

```
PLANNER_PROMPT = """
You are a Planner Agent, specialized in creating structured research ...
    plans for complex questions.

Information about research level and tools needed for the question:
{information}

Instructions:
1. Break down the research question into a series of logical steps ...
    that can be followed to find the answer.
2. Each step should be clear and actionable, guiding the research ...
    process effectively.
3. If clarity is lacking, write questions to get more information ...
    while researching.
4. Produce a multi-step plan with logical sequencing.
5. Use the information about the research level and tools needed to ...
    tailor the plan accordingly. Ignore if the information is not provided.
"""
```

B.3 Decomposer Prompt

```
DECOMPOSER_PROMPT = """
You are a question decomposition agent for multi-hop reasoning.

Given a research plan, break it down into simpler sub-questions for ...
    each tool that can be answered.

Instructions:
1. Maximum 4 sub-questions for each tool.
2. Each query should focus on ONE specific aspect.
3. Ensure that the sub-questions are logically connected and follow a ...
    clear progression towards answering the main research question.
4. Focus on factual, verifiable sub-questions.
5. Avoid overly broad or vague sub-questions.
6. Avoid overlapping sub-questions.

### Very IMPORTANT INSTRUCTIONS
You MUST call the decompose_plan tool.
Do NOT create step-wise outputs.
Only use the exact tool schema keys.
"""

USER_DECOMPOSER_PROMPT = """
Research Plan:
{plan}
"""
```

B.4 Decomposer Prompt from Iteration 2

```
DECOMPOSER_PROMPT_ITER_2 = """
You are a question decomposition agent for multi-hop reasoning.

Given a research plan and the current sub-questions, create new ...
    sub-questions to better target the information needed.

Instructions to generate new sub-questions:
{instructions}
"""

USER_DECOMPOSER_PROMPT_ITER_2 = """
Research Plan:
{plan}

Existing Sub-questions:
{existing_subquestions}
"""
```

B.5 Source Validation Prompt

```
SOURCE_VALIDATION_PROMPT = """
You are an evidence extraction and validation agent.
```

```
Your task is to analyze the provided sources and extract factual ...
    claims that are supported by each source in relation to the ...
    research question.

### Instructions:
1. Read the research question carefully.
2. Analyze each source independently.
3. Extract only claims that are directly supported or strongly implied ...
    by the source.
4. A source may:
    - support multiple claims,
    - support a single claim,
    - or support no useful claims.
5. For each extracted claim:
    - assign a confidence score (1-10) indicating how confidently the ...
        source supports the claim with respect to the research question.
    - higher score = stronger relevance + stronger evidence support.
6. Do NOT hallucinate claims that are not supported by the source.
7. Keep claims concise, factual, and evidence-oriented.
8. If no meaningful claims can be inferred from a source, return an ...
    empty claims list for that source.
"""

USER_VALIDATION_PROMPT = """
Research Question:
{question}

Sources:
{srcs}
"""
```

B.6 Source Validation Prompt from Iteration 2

```
SOURCE_VALIDATION_PROMPT_ITER2 = """
You are an evidence extraction and validation agent.

Your task is to analyze the provided sources and extract factual ...
    claims that are supported by each source in relation to the ...
    research question.

### Instructions:
1. Read the research question carefully.
2. Analyze each source independently.
3. Extract only claims that are directly supported or strongly implied ...
    by the source.
4. A source may:
    - support multiple claims,
    - support a single claim,
    - or support no useful claims.
5. For each extracted claim:
    - assign a confidence score (1-10) indicating how confidently the ...
        source supports the claim with respect to the research question.
    - higher score = stronger relevance + stronger evidence support.
6. Do NOT hallucinate claims that are not supported by the source.
7. Keep claims concise, factual, and evidence-oriented.
8. If no meaningful claims can be inferred from a source, return an ...
    empty claims list for that source.
9. Use additional instructions to generate claims
"""
```

```
USER_VALIDATION_PROMPT_ITER2 = """
Research Question:
{question}

Sources:
{srcs}

Additional Instruction:
{instructions}
"""
```

B.7 Reflection Prompt

```
REFLECTION_PROMPT = """
You are a reflection agent.

IMPORTANT:
Return ONLY actual field values in the tool call.
Do NOT return schema definitions.
Do NOT return field descriptions.
Do NOT return types.

You must populate:
- next_node
- reasoning
- instructions

Choose:
- validation_node -> when claims are poor or irrelevant
- decomposer_node -> when claims are good but more information is needed
"""

USER_REFLECTION_PROMPT = """
Query:
{query}

Research Plan:
{plan}

Validated Source Claims:
{validated_sources}
"""
```

B.8 Synthesis Prompt

```
SYNTHESIS_PROMPT_RESEARCH = """
You are a deep research synthesis agent.
Generate a comprehensive markdown report using the comprehensive ...
    information gathered.

Instructions:
1. The report should be structured with clear sections and headings.
2. Include a summary of key findings at the beginning.
3. Provide detailed explanations and reasoning for each conclusion.
4. Ensure all claims are supported by the provided evidence.
```

```
5. Use clear and precise language suitable for an academic audience.
6. Include citations for all sources of information.
7. Avoid including any information that is not directly supported by ...
   the provided evidence.
8. The report should be comprehensive and cover all aspects of the ...
   research question.
9. The report should be informative and provide valuable insights ...
   based on the evidence.
10. List all the sources used in the synthesis at the end of the report.
"""

SYNTHESIS_PROMPT_SIMPLE = """
You are a intelligent agent specialized in answering complex questions ...
   by synthesizing information.

Instructions:
1. Analyze the question and identify the key components.
2. Break down the question into smaller parts if necessary.
3. Use logical reasoning to connect the information and draw conclusions.
4. Provide a clear and concise answer to the question.
5. Ensure that the answer is directly addressing the question and is ...
   supported by logical reasoning.
"""
```

B.9 Evaluation Prompts

B.9.1 Context Relevance

```
CONTEXT_SYSTEM_PROMPT = """
You are an expert evaluator.

Evaluate how useful and relevant the retrieved context is
for answering the user question.

Evaluate:
- relevance
- usefulness
- coverage
- noise

STRICTLY use the tool.
Return:
- score (0-10)
- short reason
"""

CONTEXT_USER_PROMPT = """
Architecture:
{architecture}

User Question:
{query}

Sub Questions:
{subqueries}

Collected Sources:
{sources}
```

```
Validated Sources:
{validated_sources}

Final Answer:
{final_answer}
"""
```

B.9.2 Answer Relevance

```
ANSWER_SYSTEM_PROMPT = """
You are an expert evaluator.

Evaluate whether the final answer actually answers
the user question.

Evaluate:
- completeness
- correctness
- focus
- missing aspects

STRICTLY use the tool.
Return:
- score (0-10)
- short reason
"""

ANSWER_USER_PROMPT = """
Architecture:
{architecture}

User Question:
{query}

Final Answer:
{final_answer}
"""
```

B.9.3 Faithfulness / Groundedness

```
FAITHFULNESS_SYSTEM_PROMPT = """
You are an expert evaluator.

Evaluate whether the claims in the final answer
are supported by the retrieved evidence/context.

Evaluate:
- groundedness
- hallucinations
- unsupported claims
- evidence support

STRICTLY use the tool.
Return:
- score (0-10)
- short reason
"""
```

```
FAITHFULNESS_USER_PROMPT = """
Architecture:
{architecture}

User Question:
{query}

Collected Sources:
{sources}

Validated Sources:
{validated_sources}

Final Answer:
{final_answer}
"""
```

B.9.4 Decomposition Quality

```
DECOMPOSITION_SYSTEM_PROMPT = """
You are an expert evaluator.

Evaluate the decomposition quality.

Evaluate:
- coverage of subquestions
- redundancy
- granularity
- logical structure
- dependency quality

If decomposition does not exist,
give score 0.

STRICTLY use the tool.
Return:
- score (0-10)
- short reason
"""

DECOMPOSITION_USER_PROMPT = """
Architecture:
{architecture}

User Question:
{query}

Sub Questions:
{subqueries}

Plan:
{plan}
"""
```

B.9.5 Reflection Quality

```
REFLECTION_SYSTEM_PROMPT = """
You are an expert evaluator.

Evaluate whether reflection/self-correction
improved the reasoning pipeline.

Evaluate:
- error correction
- iterative improvement
- retrieval refinement
- answer improvement

If reflection does not exist,
give score 0.

STRICTLY use the tool.
Return:
- score (0-10)
- short reason
"""

REFLECTION_USER_PROMPT = """
Architecture:
{architecture}

User Question:
{query}

Reflection:
{reflection}

Iterations:
{loop_count}

Validated Sources:
{validated_sources}

Final Answer:
{final_answer}
"""
```

Appendix C

Tool Schemas

This appendix includes the structured schemas used for tool calling and node outputs.

C.1 Pre-Planner Tool

```
PRE_PLANNER_TOOL = {
    "type": "function",
    "function": {
        "name": "pre_plan",
        "description": "Pre-plan the research approach",
        "parameters": {
            "type": "object",
            "properties": {
                "research_required": {
                    "type": "boolean",
                    "description": "Indicates if research is required"
                },
                "reasoning_depth": {
                    "type": "enum",
                    "enum": ["shallow", "medium", "deep"],
                    "description": "The depth of reasoning needed ...
                       (shallow, medium, deep) if research is required."
                },
                "tools_needed": {
                    "type": "array",
                    "items": {
                        "type": "enum",
                        "enum": ["web_search", "arxiv", "wikipedia"]
                    },
                    "description": "The tools needed if research is ...
                       required."
                }
            },
            "required": ["research_required"],
            "additionalProperties": False
        }
    }
}
```

C.2 Decomposer Tool

```
DECOMPOSER_TOOL = {
    "type": "function",
```

```
    "function": {
        "name": "decompose_plan",
        "description": "Decompose research plan into focused ...
           subqueries for each tool",
        "parameters": {
            "type": "object",
            "properties": {
                "web_search": {
                    "type": "array",
                    "items": {
                        "type": "string"
                    },
                    "minItems": 1,
                    "description": "Subqueries for the web_search tool"
                },
                "arxiv": {
                    "type": "array",
                    "items": {
                        "type": "string"
                    },
                    "minItems": 1,
                    "description": "Subqueries for the arxiv tool"
                },
                "wikipedia": {
                    "type": "array",
                    "items": {
                        "type": "string"
                    },
                    "minItems": 1,
                    "description": "Subqueries for the wikipedia tool"
                }
            },
            "required": ["web_search", "arxiv", "wikipedia"],
            "additionalProperties": False
        }
    }
}
```

C.3 Decomposer Tool Iteration 2

```
DECOMPOSER_TOOL_ITER_2 = {
    "type": "function",
    "function": {
        "name": "decompose_plan",
        "description": "Decompose research plan into focused ...
           subqueries using instructions from reflection",
        "parameters": {
            "type": "object",
            "properties": {
                "web_search": {
                    "type": "array",
                    "items": {
                        "type": "string"
                    },
                    "minItems": 1
                },
                "arxiv": {
                    "type": "array",
                    "items": {
```

```
                        "type": "string"
                    },
                    "minItems": 1
                },
                "wikipedia": {
                    "type": "array",
                    "items": {
                        "type": "string"
                    },
                    "minItems": 1
                }
            },
            "required": ["web_search", "arxiv", "wikipedia"],
            "additionalProperties": False
        }
    }
}
```

C.4 Source Validation Tool

```
SOURCE_VALIDATION_TOOL = {
    "type": "function",
    "function": {
        "name": "validate_source_claims",
        "description": (
            "Extract claims supported by the provided sources and ...
               assign "
            "a confidence score for how strongly each source supports "
            "the claim with respect to the research question."
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "claims": {
                    "type": "array",
                    "items": {
                        "type": "object",
                        "properties": {
                            "claim": {
                                "type": "string"
                            },
                            "confidence_score": {
                                "type": "integer",
                                "minimum": 1,
                                "maximum": 10
                            },
                            "source_id": {
                                "type": "integer"
                            },
                            "justification": {
                                "type": "string"
                            }
                        },
                        "required": [
                            "claim",
                            "confidence_score",
                            "source_id",
                            "justification"
                        ],
```

```
                        "additionalProperties": False
                    }
                }
            },
            "required": ["claims"],
            "additionalProperties": False
        }
    }
}
```

C.5 Source Validation Tool Iteration 2

```
SOURCE_VALIDATION_TOOL_ITER2 = {
    "type": "function",
    "function": {
        "name": "validate_source_claims",
        "description": (
            "Extract claims supported by the provided sources and ...
               assign "
            "a confidence score using additional instructions."
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "claims": {
                    "type": "array",
                    "items": {
                        "type": "object",
                        "properties": {
                            "claim": {
                                "type": "string"
                            },
                            "confidence_score": {
                                "type": "integer",
                                "minimum": 1,
                                "maximum": 10
                            },
                            "source_id": {
                                "type": "integer"
                            },
                            "justification": {
                                "type": "string"
                            }
                        },
                        "required": [
                            "claim",
                            "confidence_score",
                            "source_id",
                            "justification"
                        ],
                        "additionalProperties": False
                    }
                }
            },
            "required": ["claims"],
            "additionalProperties": False
        }
    }
}
```

C.6 Reflection Tool

```
REFLECTION_TOOL = {
    "type": "function",
    "function": {
        "name": "reflect_on_claims",
        "description": "Determine the next workflow node based on ...
           validated claims.",
        "parameters": {
            "type": "object",
            "properties": {
                "next_node": {
                    "type": "string",
                    "enum": [
                        "validation_node",
                        "decomposer_node"
                    ]
                },
                "reasoning": {
                    "type": "string"
                },
                "instructions": {
                    "type": "string"
                }
            },
            "required": [
                "next_node",
                "reasoning",
                "instructions"
            ]
        }
    }
}
```

C.7 Dynamic Evaluation Tool

The evaluation tool schema is dynamically generated based on the evaluation metric being measured.
Different evaluation metrics such as:

- Context Relevance
- Answer Relevance
- Faithfulness / Groundedness
- Decomposition Quality
- Reflection Quality

use the same underlying schema structure but adapt:

- tool name
- metric description

- scoring criteria

This enables reusable and modular evaluation across multiple architectures and reasoning stages.

```
{
    "type": "function",
    "function": {
        "name": tool_name,
        "description": metric_description,
        "parameters": {
            "type": "object",
            "properties": {
                "score": {
                    "type": "integer",
                    "minimum": 0,
                    "maximum": 10,
                    "description": f"{metric_name} score from 0 to 10"
                },
                "reason": {
                    "type": "string",
                    "description": "Very short reason for the score"
                }
            },
            "required": ["score", "reason"],
            "additionalProperties": False
        }
    }
}
```

Appendix D

Langfuse Logs and Traces

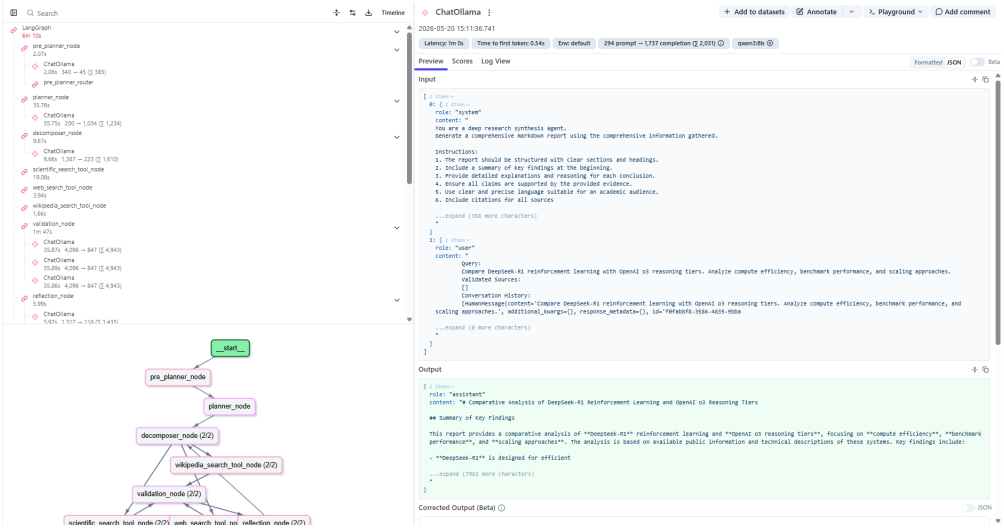

Figure D.1: Langfuse UI for observability.

Figure D.2: Agent execution flow visualized in the Langfuse UI.

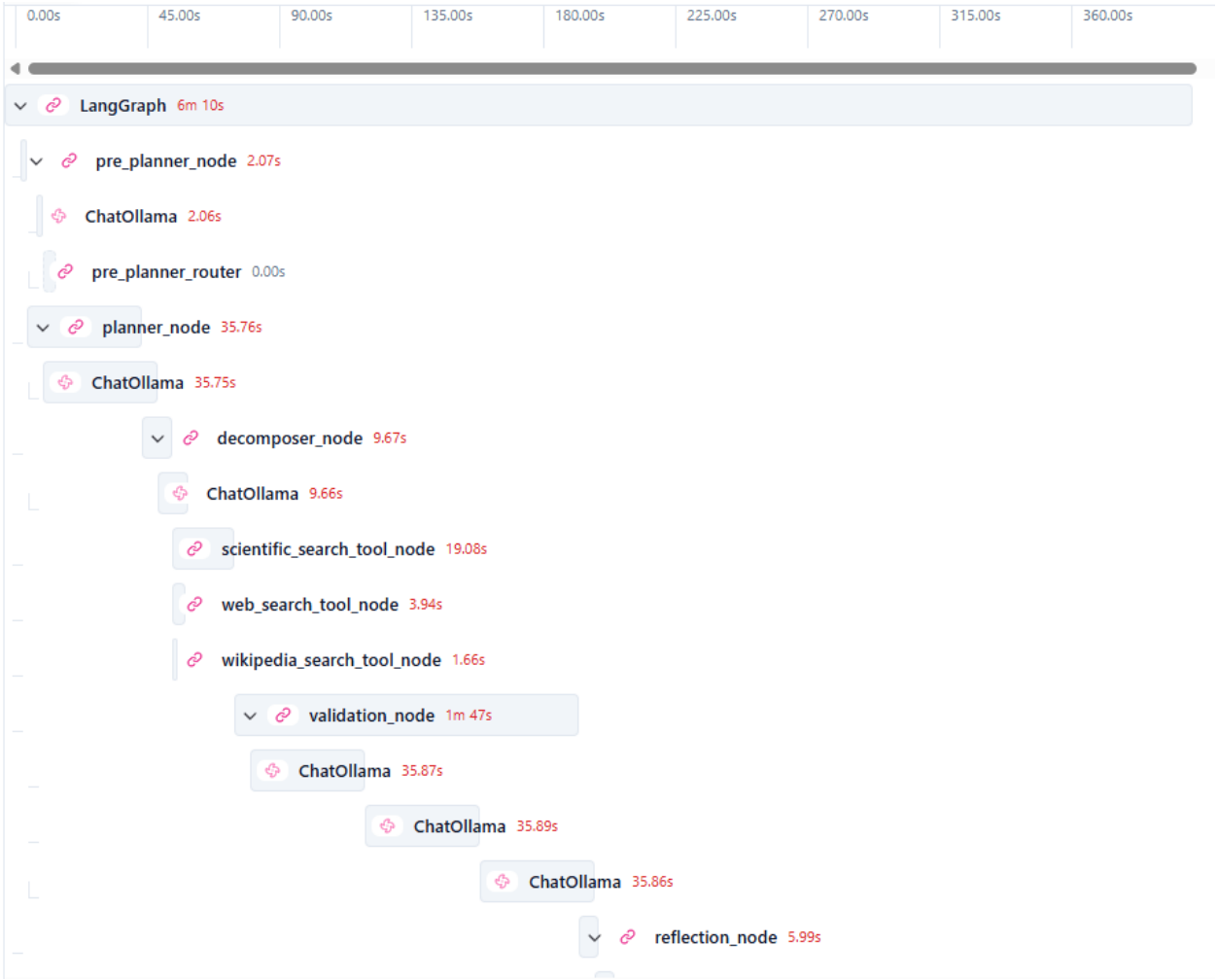

Figure D.3: Node execution timelines and durations visualized in the Langfuse UI.

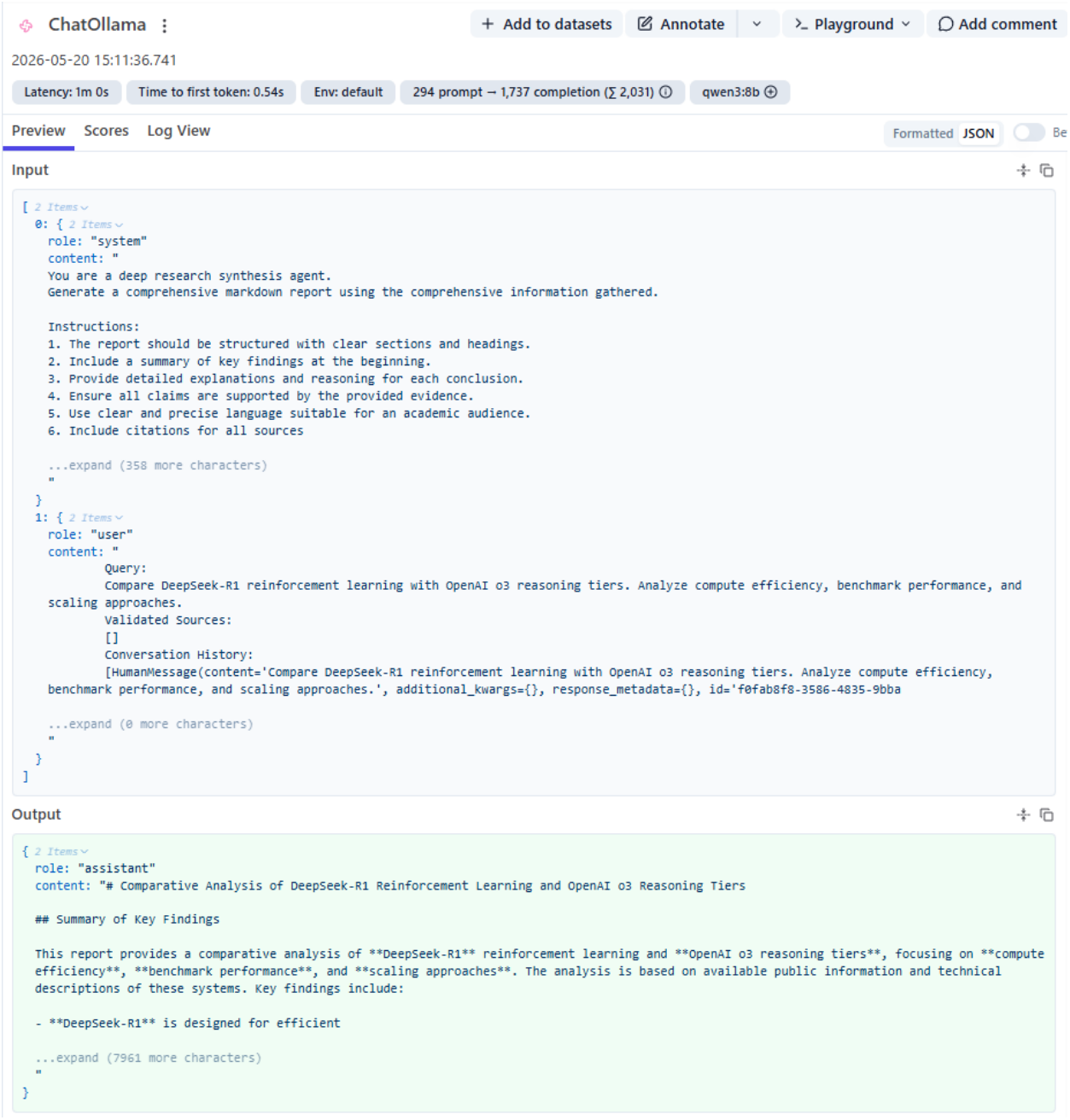

Figure D.4: Input and output of each agent component visualized in the Langfuse UI.

Appendix E

Evaluation Questions

This appendix includes the evaluation questions used to compare different architectures.

1. Investigate the performance-to-compute efficiency of Test-Time Scaling (System 2 thinking) versus traditional Pre-training Scaling Laws. Specifically, compare the cost-effectiveness of DeepSeek-R1's reinforcement learning approach against OpenAI o3's reasoning tiers for mathematical reasoning tasks.
2. Analyze the 'Lost-in-the-Middle' phenomenon in 2026's ultra-long context models (1M+ tokens). Compare the retrieval accuracy of Meta Llama 4 Scout (10M token window) against hybrid Mamba-Transformer (Jamba) architectures.
3. Evaluate the shift from Late Fusion to Early Fusion in 2026 Vision-Language Models (VLMs). Compare the visual grounding capabilities of Gemini 2.5 Pro and InternVL3-78B on the MMMU (Massive Multi-discipline Multimodal Understanding) benchmark.